
SMART STREET

Systeme de Gestion Intelligente de la Circulation Urbaine

Rapport Technique

Projet : **Smart Street**

Equipe	Loujayn Omri · Salma Mhiri · Sima Rached · Khalil Ben Ghanem
Modules	Radar · Feu Tricolore · Robot Voiture · Manette · Parking
Les cartes enjeu	ESP32-CAM · ESP8266 · ESP32 · Arduino UNO
Communication	ESP-NOW · WiFi AP · HTTP

Table des matières

1	Introduction et Contexte	5
1.1	Présentation du projet Smart Street	5
1.2	Inventaire global des capteurs et actionneurs	6
1.3	Architecture de communication inter-modules	7
1.4	Objectifs du projet	7
1.5	Philosophie d'implémentation	7
1.6	Dependances logicielles	8
2	Diagrammes d'Activité et de Séquence	9
2.1	Diagramme d'activite global	10
2.2	Diagramme de sequence – Detection et enregistrement photo	11
2.3	Diagramme de séquence – Robot Voiture	11
3	Module 1 – Feu Tricolore + street lamps	12
3.1	Présentation et rôle	12
3.2	Capteurs utilises	12
3.2.1	Capteur de lumière – LDR (Photorésistance)	12
3.2.2	Bouton poussoir – Appel piéton	13
3.3	Actionneurs utilisés	13
3.3.1	LEDs de signalisation tricolore	13
3.4	Emission ESP-NOW vers le module Radar	15
3.5	Schema de câblage	16
3.6	State Machine/loop	17
4	Module 2 – Radar	18
4.1	Presentation et role	18
4.2	Capteurs utilises	18
4.2.1	Capteur PIR – HC-SR501	18
4.2.2	Reception ESP-NOW – état du feu	19
4.3	Actionneurs utilises	19
4.3.1	Servomoteur SG90	19
4.3.2	Camera OV2640 – Capture JPEG	20
4.3.3	Serveur web HTTP embarque	21
4.3.4	loop Radar	21
4.4	Schema de câblage	23

4.5	State Machine – Radar	23
5	Module 3 – Manette	24
5.1	Presentation et role	24
5.2	Capteurs utilises	24
5.2.1	Module Joystick analogique (VRX, VRY, SW)	24
5.3	Code de manette	24
5.3.1	Explication du code	26
5.4	State Machine – Manette	27
6	Module 4 – Robot Voiture	28
6.1	Presentation et role	28
6.2	Capteurs utilises	28
6.3	Actionneurs utilises	28
6.3.1	Pont en H L298N – Commande moteurs DC	28
6.4	Code voiture	29
6.4.1	Explication du code	32
6.5	Schema de câblage	34
6.6	State Machine – Robot Voiture	34
7	Module 5 – Parking	35
7.1	Presentation et rôle	35
7.2	Capteurs utilisés	35
7.2.1	Capteur ultrason HC-SR04 (x2)	35
7.3	Actionneurs utilisés	36
7.3.1	Servo – Barriere d'accès	36
7.3.2	Afficheur LCD I2C 16x2	36
7.3.3	Buzzer – Alarme parking complet	37
7.3.4	LED indicateur	37
7.4	Schema de câblage	37
7.5	Logique de detection entree/sortie et State Machine	38
7.6	Code complet	38
8	Conclusion	43

Table des figures

1.1	Architecture de communication inter-modules Smart Street	7
2.1	Diagramme d'activite – Detection infraction	10

2.2	Diagramme de sequence de Detection d’infraction et enregistrement photo	11
2.3	Diagramme de sequence de Telecommande Robot Voiture via ESP-NOW	11
3.1	Schema de cablage – Feu + street lights(ESP8266 + Leds + feu tricolore + bouton poussoir + LDR + transistor 2N2222 + resistance 1Kohm) .	16
3.2	FSM – Module Feu Tricolore (superloop bloquante)	17
4.1	Schema de cablage – Radar (ESP32-CAM + PIR HC-SR501 + Servo SG90)	23
4.2	FSM	23
5.1	FSM	27
6.1	Schema de cablage – Robot Voiture (ESP8266 + L298N + Moteurs DC)	34
6.2	FSM	34
7.1	Schema de cablage – Module Parking (Arduino UNO + HC-SR04 x2 + Servo + LCD + Buzzer + LED)	37
7.2	FSM – Module Parking (Arduino UNO)	38

Liste des tableaux

1.1	Recapitulatif Capteurs / Actionneurs du systeme Smart Street	6
1.2	Bibliotheques et outils logiciels – systeme complet	8
3.1	Caracteristiques LDR – Module Feu Tricolore	12
3.2	Caracteristiques bouton piéton	13
3.3	LEDs de signalisation – Brochage et etat logique	13
4.1	Caracteristiques PIR HC-SR501	18
4.2	Caracteristiques Servo SG90 – Radar	20
4.3	Caracteristiques Camera OV2640	20
4.4	Cablage complet – Module Radar	23
5.1	Caracteristiques Joystick analogique – Manette	24
5.2	Cablage – Manette ESP32	27
6.1	Caracteristiques L298N et logique de commande	28
6.2	Cablage – Robot Voiture ESP8266 / L298N	34
7.1	Caracteristiques HC-SR04	35
7.2	Caracteristiques Servo barriere – Parking	36

7.3	Caracteristiques LCD I2C	37
7.4	Câblage complet – Module Parking Arduino UNO	38

1. Introduction et Contexte

1.1 Présentation du projet Smart Street

Le projet **Smart Street** est un système embarqué distribué qui simule une infrastructure urbaine intelligente. Il intègre une chaîne complète de **capteurs** (détection de mouvement, présence, lumière, distance, joystick analogique) et d'**actionneurs** (moteurs DC, servomoteurs, LEDs, buzzer, afficheur LCD, caméra) interconnectés par les protocoles **ESP-NOW** et **HTTP/WiFi**.

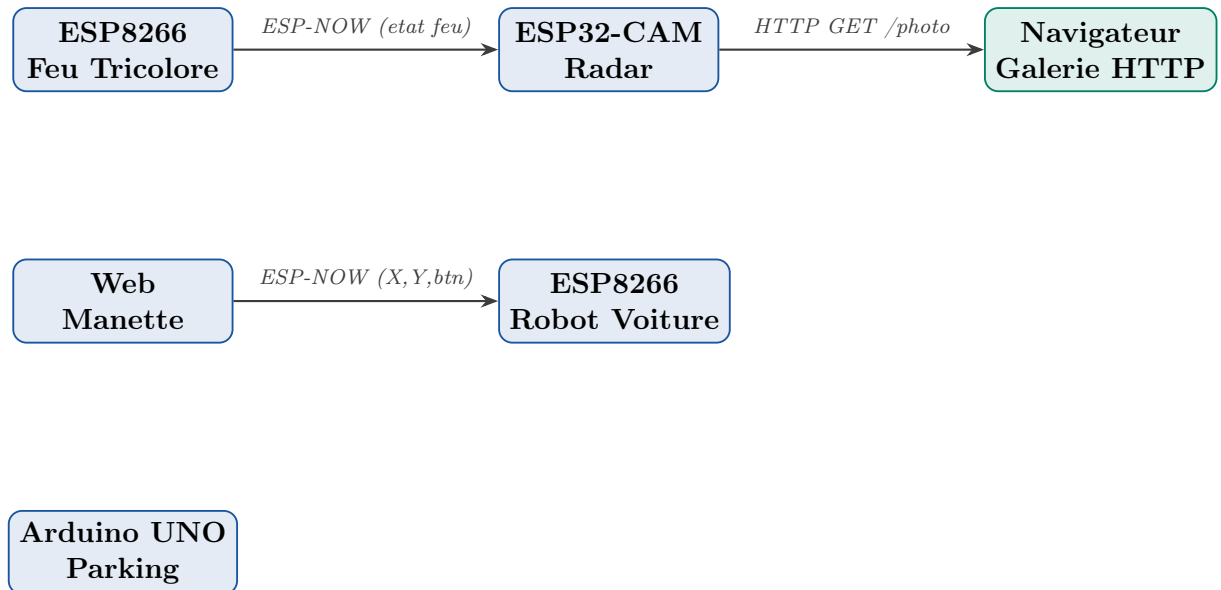
Ce projet constitue une application directe des concepts fondamentaux du cours **Capteurs & Actionneurs** : acquisition de signaux physiques, conditionnement, conversion analogique-numérique, commande d'actionneurs et gestion temps-réel.

1.2 Inventaire global des capteurs et actionneurs

TABLE 1.1 – Recapitulatif Capteurs / Actionneurs du systeme Smart Street

Module	Composant	Type	Signal	Role
Feu	LDR	Capteur	Analogique	Mesure luminosité ambiante
Feu	Bouton piéton	Capteur	Numérique	Appui demande de passage
Feu	LEDs tricolores	Actionneur	Numerique	Signalisation voiture/-piéton
Feu	LEDs nocturnes	Actionneur	Numérique	Eclairage adaptatif
Radar	PIR HC-SR501	Capteur	Numérique	Detection infrarouge passif
Radar	Servo SG90	Actionneur	PWM	Orientation caméra (panoramique)
Radar	Camera OV2640	Actionneur	SCCB/DVP	Capture image JPEG
Radar	Flash LED	Actionneur	Numérique	Eclairage capture
Manette	Joystick VRX/Y	Capteur	Analogique	Position X/Y (ADC 12 bits)
Manette	Bouton SW	Capteur	Numerique	Clic toggle activation
Voiture	Moteur DC x2	Actionneur	PWM	Propulsion différentielle
Voiture	Pont H L298N	Actionneur	PWM+DIR	Commande direction/-vitesse
Parking	HC-SR04 x2	Capteur	Ultrason	Détection présence véhicule
Parking	Servo barrière	Actionneur	PWM	Controle accès
Parking	LCD I2C 16x2	Actionneur	I2C	Affichage places disponibles
Parking	Buzzer	Actionneur	Numérique	Alerte parking complet
Parking	LED indicateur	Actionneur	PWM	Indication présence capteur

1.3 Architecture de communication inter-modules



Autonome (LCD I2C + Serie)

FIGURE 1.1 – Architecture de communication inter-modules Smart Street

1.4 Objectifs du projet

- F1.** Gérer automatiquement le cycle de feux avec prise en compte du piéton et de la luminosité.
- F2.** Détecter les infractions au feu rouge par capteur PIR et photographier les contrevenants.
- F3.** Exposer les photos d'infraction via un serveur web embarqué accessible en WiFi.
- F4.** Piloter sans fil un robot voiture par joystick analogique via ESP-NOW.
- F5.** Gérer automatiquement l'accès au parking par détection ultrason, avec affichage et alarme.

1.5 Philosophie d'implémentation

Tous les modules reposent sur le même paradigme logiciel : une `loop()` combinée à des **machines à états finis (FSM)** explicites. Les tâches longues (capture, HTTP) sont non-bloquantes grâce à `millis()`.

1.6 Dependances logicielles

TABLE 1.2 – Bibliothèques et outils logiciels – système complet

Bibliothèque	Module	Version	Usage
Arduino IDE / PlatformIO	Tous	≥ 2.0	Environnement de dev
esp32 board package	Radar, Manette	≥ 2.0	Support Arduino ESP32
esp8266 board package	Feu, Voiture	≥ 3.1	Support Arduino ESP8266
ESP32Servo	Radar	≥ 0.13	PWM servo SG90
esp_camera	Radar	(SDK inclus)	Capture OV2640
WebServer	Radar	(SDK inclus)	HTTP embarque
espnw / esp_now	Tous WiFi	(SDK inclus)	Communication sans fil
LiquidCrystal_I2C	Parking	$\geq 1.1.2$	Afficheur LCD I2C
Servo (Arduino)	Parking	(SDK inclus)	Barrière SG90
Flask	(serveur)	≥ 3.0	Dashboard optionnel PC

2. Diagrammes d'Activité et de Séquence

2.1 Diagramme d'activite global

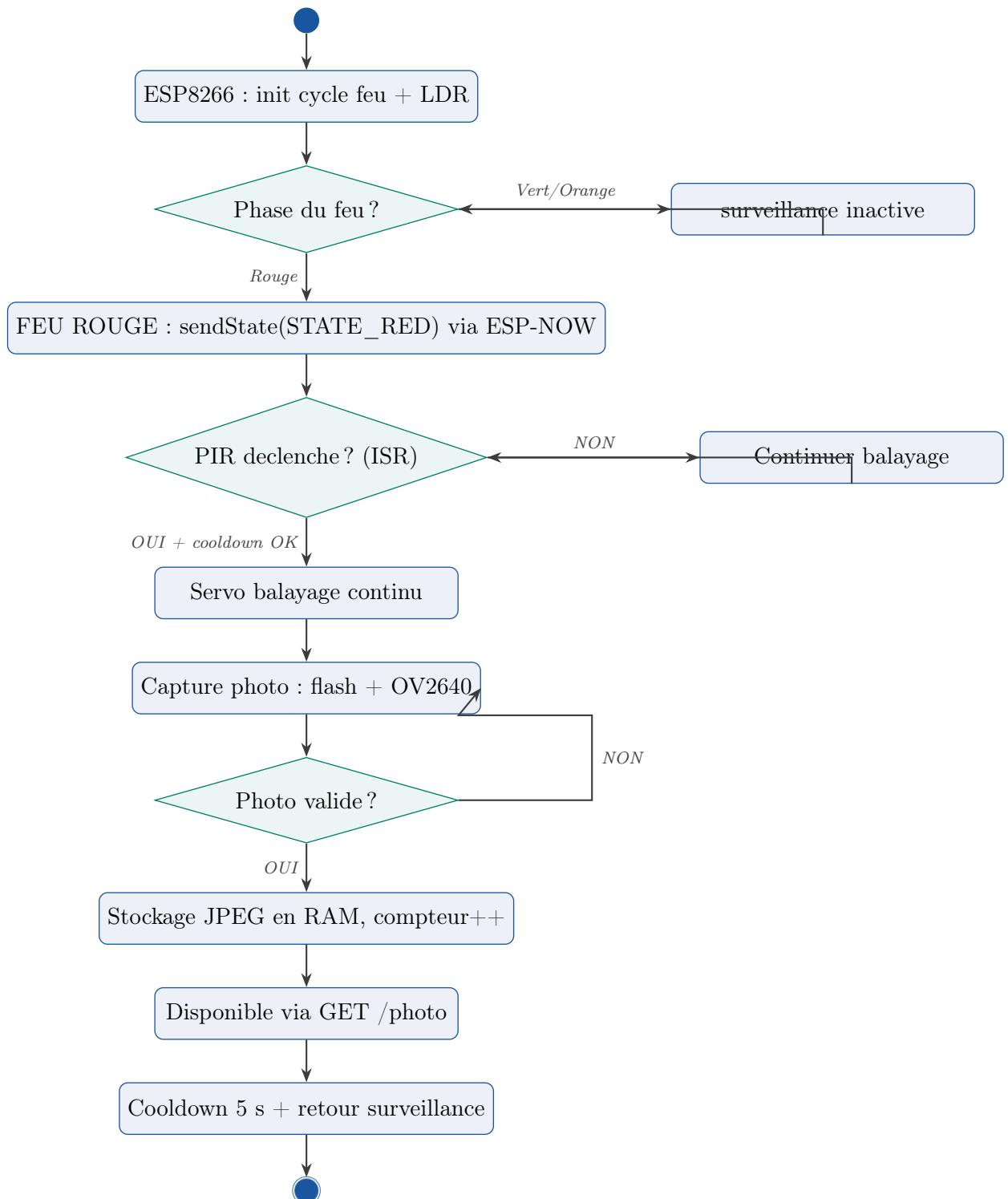


FIGURE 2.1 – Diagramme d'activite – Detection infraction

2.2 Diagramme de sequence – Detection et enregistrement photo

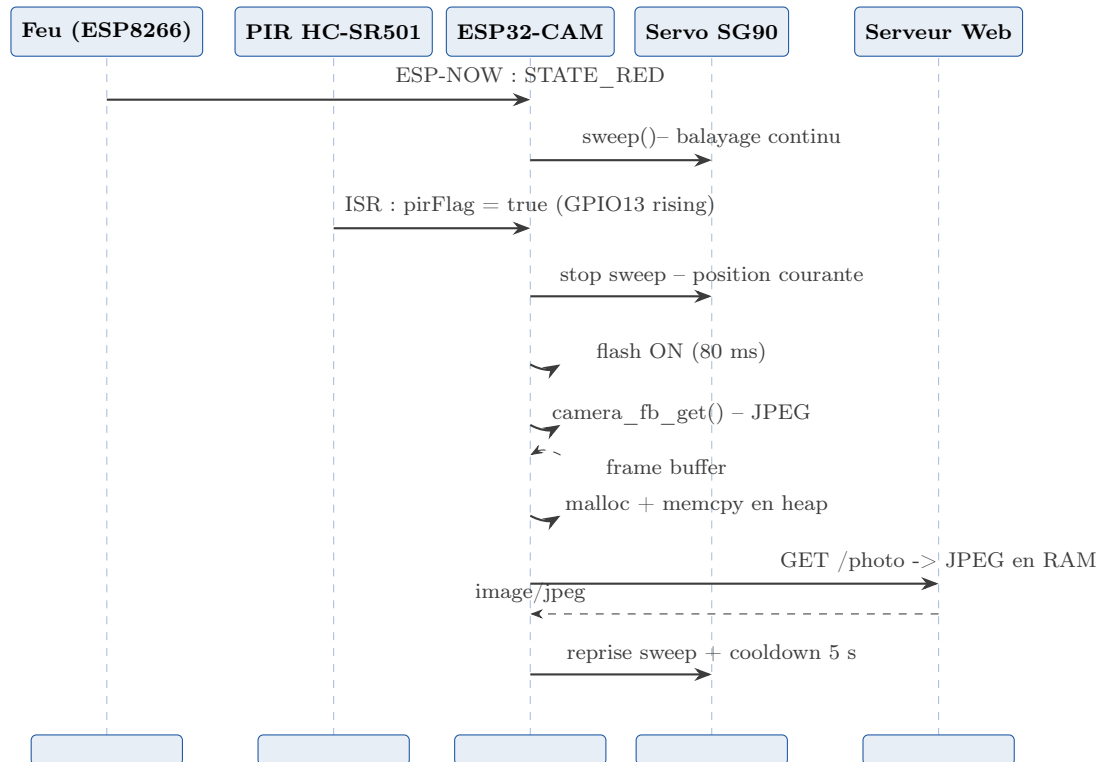


FIGURE 2.2 – Diagramme de sequence de Detection d’infraction et enregistrement photo

2.3 Diagramme de séquence – Robot Voiture

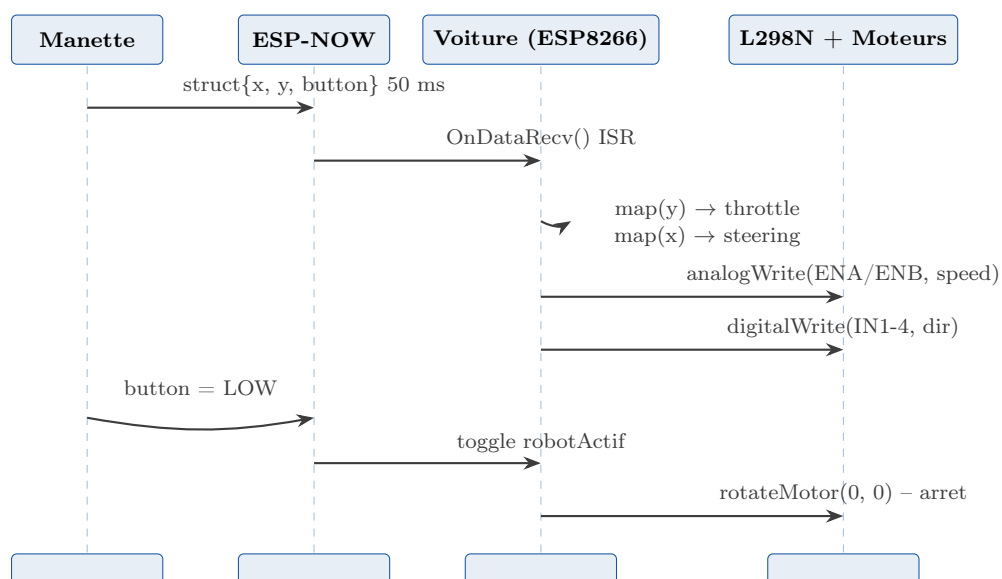


FIGURE 2.3 – Diagramme de sequence de Telecommande Robot Voiture via ESP-NOW

3. Module 1 – Feu Tricolore + street lamps

3.1 Présentation et rôle

Le module **Feu Tricolore** est la signalisation du carrefour. Il execute le cycle de feux voiture/pieton, gere le bouton d'appel pieton, active un eclaireage nocturne adaptatif via un capteur de lumiere, et transmet l'etat courant du feu au module radar via le protocole **ESP-NOW**.

3.2 Capteurs utilises

3.2.1 Capteur de lumière – LDR (Photorésistance)

Capteur : La LDR (*Light Dependent Resistor*) est un capteur **passif resistif** dont la resistance varie de plusieurs $M\Omega$ (obscurite) a quelques centaines d' Ω (plein soleil). Elle est lue via le **convertisseur analogique-numerique (ADC)** du port A0 du NodeMCU ESP8266 (10 bits, 0-1023).

TABLE 3.1 – Caracteristiques LDR – Module Feu Tricolore

Parametre	Valeur / Description
Type	Resistif passif (photoresistance CdS)
Plage resistance	1 k Ω (lumiere) – 1 M Ω (nuit)
Interface	Diviseur de tension + ADC A0 (10 bits)
Seuil numerique	> 550 sur 1023 → activation eclaireage nocturne
Tension lecture	3.3 V (NodeMCU) via diviseur resistif

```
1 const int ldrPin      = A0;      // ADC 10 bits (0-1023)
2 const int nightLeds = 16;      // GPIO16 (D0)
3
4 void updateNightLeds() {
5     int ldrValue = analogRead(ldrPin); // lecture ADC
6     // Seuil : > 550 = faible luminosite -> allumer LEDs
7     if (ldrValue > 550) digitalWrite(nightLeds, HIGH);
8     else                  digitalWrite(nightLeds, LOW);
9 }
```

Listing 3.1 – Lecture LDR et commande eclaireage nocturne

3.2.2 Bouton poussoir – Appel piéton

Capteur : Le bouton est un capteur *numerique passif*. L'ESP8266 le lit en mode *INPUT_PULLUP* : l'entree est naturellement a *HIGH* (3.3 V via resistance interne) et passe a *LOW* quand le pieton appuie.

TABLE 3.2 – Caracteristiques bouton pieton

Parametre	Valeur
Type	Contact NO (Normalement Ouvert)
Interface	GPIO2 (D4), mode INPUT_PULLUP
Etat repos	HIGH (3.3 V via pull-up interne)
Etat actif	LOW (0 V, court-circuit vers GND)
Action	Interrompre le feu vert apres 1 s de garde

3.3 Actionneurs utilisés

3.3.1 LEDs de signalisation tricolore

Actionneur : Chaque LED est commandee par une sortie GPIO en mode *OUTPUT*. Une resistance serie de 220 Ω limite le courant a environ 10–15 mA.

TABLE 3.3 – LEDs de signalisation – Brochage et etat logique

LED	GPIO	NodeMCU	Etat actif	Condition d'activation
Verte (traffic)	GPIO13	D7	HIGH	Phase verte
Orange (traffic)	GPIO12	D6	HIGH	Phase transition
Rouge (traffic)	GPIO14	D5	HIGH	Phase rouge
Verte (pieton)	GPIO5	D1	HIGH	Phase rouge voiture (pieton passe)
Rouge (pieton)	GPIO4	D2	HIGH	Phase verte voiture (pieton stoppe)
LEDs nocturnes	GPIO16	D0	HIGH	LDR > 550 (luminosite faible)

```

1 void redState() {
2   digitalWrite(carRed, HIGH); digitalWrite(carGreen, LOW);
3   digitalWrite(carOrange, LOW); digitalWrite(pedGreen, HIGH);
4   digitalWrite(pedRed, LOW);
5   sendState(STATE_RED);           // notification ESP-NOW
6 }
7 void greenState() {
8   digitalWrite(carGreen, HIGH);

```

```

9   digitalWrite(carOrange, LOW);
10  digitalWrite(carRed,    LOW);
11  digitalWrite(pedGreen,  LOW);
12  digitalWrite(pedRed,    HIGH);
13  sendState(STATE_GREEN);
14  }
15
16  void blinkOrange() {
17    digitalWrite(carGreen, LOW);
18    digitalWrite(carRed,   LOW);
19    sendState(STATE_ORANGE);
20
21    for (int i = 0; i < 10; i++) {
22      digitalWrite(carOrange, HIGH); delay(200);
23      digitalWrite(carOrange, LOW);  delay(200);
24    }
25  }
26  void loop() {
27    updateNightLeds();
28    // red for cars
29    Serial.println("\nPhase 1: FEU ROUGE (15 secondes)");
30    redState();
31    for (int i = 0; i < 150; i++) {    // 150    100ms = 15s
32      updateNightLeds();
33      delay(100);
34    }
35
36    // green for cars
37    Serial.println("\nPhase 2: FEU VERT (15 secondes max)");
38    greenState();
39    for (int i = 0; i < 150; i++) {
40      updateNightLeds();
41      delay(100);
42      if (i > 10) {
43        if (digitalRead(buttonPin) == LOW) break;
44      }
45    }
46
47    // orang
48    Serial.println("\nPhase 3: FEU ORANGE");
49    blinkOrange();
50    Serial.println("ANLOG");
51    Serial.println(analogRead(A0));
52    delay(500);
53  }

```

Listing 3.2 – Superloop Feu Tricolore avec LDR et bouton piéton

3.4 Emission ESP-NOW vers le module Radar

L'ESP8266 est le **contrôleur ESP-NOW** : il envoie l'état du feu à l'ESP32-CAM à chaque changement de phase. La structure partagée est identique des deux cotes.

```

1 typedef enum { STATE_RED, STATE_GREEN, STATE_ORANGE } TrafficState;
2 typedef struct { TrafficState state; } struct_message;
3
4 uint8_t camMAC[] = {0x00, 0x4B, 0x12, 0x95, 0x3E, 0x1C};
5
6 struct_message dataToSend;
7
8 void onDataSent(uint8_t *mac_addr, uint8_t status) {
9     Serial.print("[SENT] ");
10    Serial.println(status == 0 ? "OK" : "FAIL");
11 }
12
13 //      FONCTION POUR AFFICHER L' TAT
14 String getStateString(TrafficState state) {
15     switch(state) {
16         case STATE_RED:
17             return "ROUGE";
18         case STATE_GREEN:
19             return "VERT";
20         case STATE_ORANGE:
21             return "ORANGE";
22         default:
23             return "INCONNU";
24     }
25 }
26
27 void sendState(TrafficState s) {
28     dataToSend.state = s;
29     Serial.println(getStateString(s));
30     esp_now_send(camMAC, (uint8_t *)&dataToSend, sizeof(dataToSend));
31 }
32
33 void setup() {
34     Serial.begin(115200);
35
36     pinMode(carGreen, OUTPUT);
37     pinMode(carOrange, OUTPUT);
38     pinMode(carRed, OUTPUT);
39     pinMode(pedGreen, OUTPUT);
40     pinMode(pedRed, OUTPUT);
41     pinMode(buttonPin, INPUT_PULLUP);
42     pinMode(nightLeds, OUTPUT);
43
44     WiFi.mode(WIFI_STA);
45     WiFi.disconnect();
46

```



```

47 // On force le canal 1 (car la CAM a cr son AP sur le canal 1)
48 wifi_promiscuous_enable(1);
49 wifi_set_channel(1);
50 wifi_promiscuous_enable(0);
51
52 if (esp_now_init() != 0) return;
53
54 esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
55 esp_now_register_send_cb(onDataSent);
56 esp_now_add_peer(camMAC, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
57
58 Serial.println("ESP8266 pret sur Canal 1");
59 }

```

Listing 3.3 – Structure et envoi ESP-NOW – Feu Tricolore

3.5 Schema de câblage

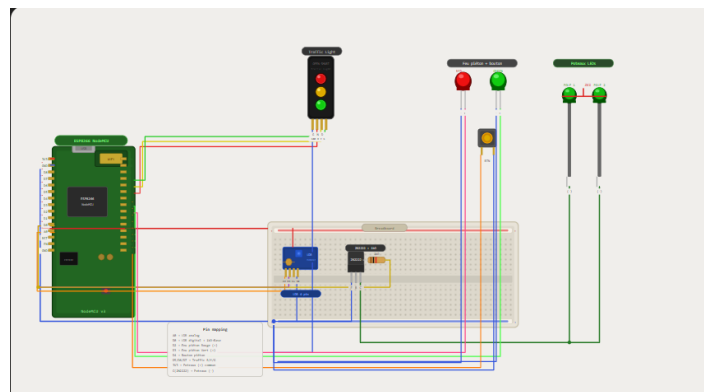


FIGURE 3.1 – Schema de câblage – Feu + street lights(ESP8266 + Leds + feu tricolore + bouton poussoir + LDR + transistor 2N2222 + resistance 1Kohm)

3.6 State Machine/loop

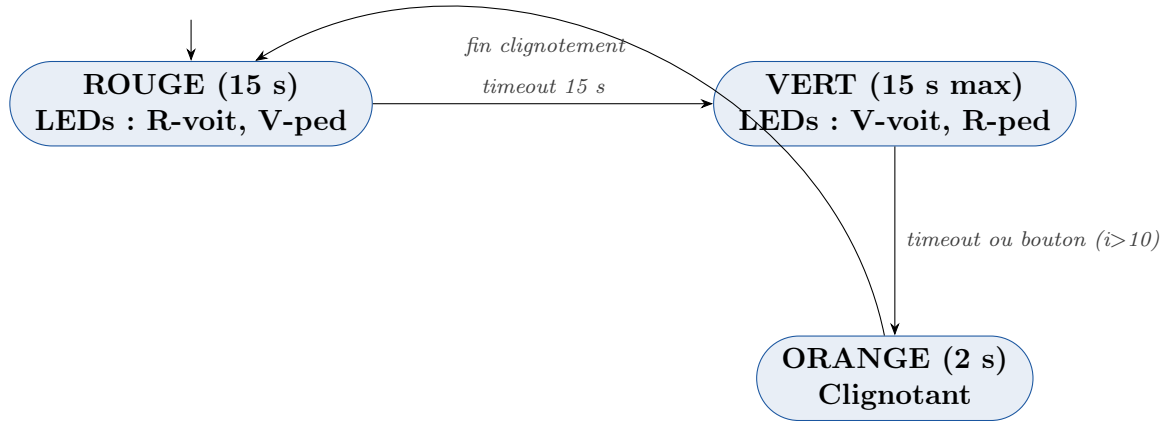


FIGURE 3.2 – FSM – Module Feu Tricolore (superloop bloquante)

4. Module 2 – Radar

4.1 Presentation et role

Le module **Radar** surveille le carrefour. Il recoit l'état du feu via ESP-NOW, detecte tout mouvement par capteur PIR, oriente sa camera par servomoteur en balayage automatique, capture une photo JPEG lors d'une infraction (feu rouge + mouvement), et la rend disponible via un **serveur web HTTP**.

4.2 Capteurs utilises

4.2.1 Capteur PIR – HC-SR501

Capteur : *Le PIR (Passive InfraRed) est un capteur **thermique passif** : il detecte les variations du rayonnement infrarouge emis par les corps chauds en mouvement. Il ne necessite aucune source lumineuse et produit un signal **numerique TTL**. Il est connecte en interruption materielle (**attachInterrupt**) pour une latence minimale.*

TABLE 4.1 – Caracteristiques PIR HC-SR501

Parametre	Valeur
Type	Capteur infrarouge passif (pyroelectrique)
Tension alim.	5–12 V DC
Signal sortie	TTL 3.3 V (compatible ESP32 directement)
Courant veille	< 50 μ A
Angle detection	110°
Portee	3–7 m (reglable par potentiometre)
Temps retrigger	0.3–25 s (reglable)
Interface	GPIO13, interruption RISING
Mode utilise	ISR (Interrupt Service Routine)

```
1 #define PIR_PIN 13
2 volatile bool pirFlag = false;
3
4 // ISR : rapide, stockee en RAM (IRAM_ATTR)
5 void IRAM_ATTR onPIR() { pirFlag = true; }
6
```

```

7 void setup() {
8   pinMode(PIR_PIN, INPUT);
9   attachInterrupt(digitalPinToInterrupt(PIR_PIN), onPIR, RISING);
10 }
11
12 void loop() {
13   if (pirFlag) {           // consommation du flag dans la loop
14     pirFlag = false;
15     if (isRed) captureInfraction();
16   }
17 }

```

Listing 4.1 – Configuration PIR en interruption ISR

Attention : Le flag *pirFlag* est declare *volatile* pour empecher le compilateur de l'optimiser. L'ISR doit etre la plus courte possible (pas de *delay*).

4.2.2 Reception ESP-NOW – état du feu

L'ESP32-CAM recoit le paquet ESP-NOW de l'ESP8266 et met à jour le booleen *isRed*.

```

1 volatile bool isRed = false;
2
3 void onDataRecv(const esp_now_recv_info*, const uint8_t* data, int len
4   ) {
5   struct_message msg;
6   memcpy(&msg, data, sizeof(msg));
7   isRed = (msg.state == STATE_RED);
8 }

```

Listing 4.2 – Reception ESP-NOW et mise à jour état feu

4.3 Actionneurs utilises

4.3.1 Servomoteur SG90

Actionneur : Le SG90 est un *servomoteur a position* commande par signal **PWM** (50 Hz, largeur d'impulsion 500–2400 $\mu s = 0-180^\circ$). Il positionne la camera en balayage continu entre 60° et 120° pendant la surveillance, garantissant une couverture angulaire maximale du carrefour.

TABLE 4.2 – Caracteristiques Servo SG90 – Radar

Parametre	Valeur
Type	Servomoteur a position (potentiometre interne)
Tension alim.	4.8–6 V (source externe obligatoire)
Courant max	700 mA (pic au demarrage)
Couple	1.8 kg·cm (4.8 V)
Plage angulaire	0°–180°
Signal commande	PWM 50 Hz, 500–2400 μ s
Interface	GPIO14, bibliotheque ESP32Servo
Plage balayage	60° $\leftrightarrow$ 120° (pas 1°/50ms)
Position capture	Fixe pendant <code>captureDelay</code> (5 s)

```

1 Servo myServo;
2 int angle      = 90;
3 int direction = 1;           // +1 ou -1
4 unsigned long lastServoMillis = 0;
5 const int servoInterval = 50; // 50 ms entre chaque pas de 1 degre
6 const int SERVO_MIN = 60, SERVO_MAX = 120;
7
8 // Dans loop() -- non bloquant (pas de delay)
9 if (millis() - lastServoMillis >= servoInterval) {
10     lastServoMillis = millis();
11     angle += direction;
12     myServo.write(angle);
13     if (angle >= SERVO_MAX || angle <= SERVO_MIN) direction *= -1;
14 }

```

Listing 4.3 – Balayage servo en superloop non-bloquant

4.3.2 Camera OV2640 – Capture JPEG

Actionneur : L'OV2640 est un capteur d'image 2 MP integre a l'ESP32-CAM. Bien qu'etant physiquement un capteur optique, il joue ici le role d'un **actionneur de capture** : son declenchement est pilote par le firmware sur critere logique (feu rouge + PIR). La sortie est un buffer JPEG transfere en RAM.

TABLE 4.3 – Caracteristiques Camera OV2640

Parametre	Valeur
Resolution max	1600×1200 (UXGA)
Format utilise	JPEG, VGA (640×480)
Qualite JPEG	10 (0=max qualite)
Bus	SCCB (I2C-like) + DVP 8 bits
Buffer	1 frame (fb_count = 1)
Flash associe	LED GPIO4, 80 ms avant capture

```

1  const unsigned long captureDelay = 5000;  // cooldown anti-spam
2
3  void captureInfraction() {
4      if (infraBuffer != nullptr) { free(infraBuffer); infraBuffer =
        nullptr; }
5
6      digitalWrite(FLASH_GPIO, HIGH);  // flash ON
7      delay(80);  // stabilisation lumiere
8
9      camera_fb_t* fb = esp_camera_fb_get();
10
11     delay(120);
12     digitalWrite(FLASH_GPIO, LOW);  // flash OFF (200 ms total)
13
14     if (!fb) { addLog("ERREUR: capture echouee."); return; }
15
16     // Copie en heap (le buffer camera sera libere)
17     infraBuffer = (uint8_t*)malloc(fb->len);
18     if (infraBuffer) {
19         memcpy(infraBuffer, fb->buf, fb->len);
20         infraLen = fb->len; infraReady = true; infraCount++;
21     }
22     esp_camera_fb_return(fb);  // liberer le buffer DMA
23 }

```

Listing 4.4 – Capture infraction avec flash et cooldown 5 s

4.3.3 Serveur web HTTP embarque

Les photos sont exposees via un serveur HTTP integre a l'ESP32-CAM. Un dashboard permet de superviser l'etat et de tester les capteurs.

```

1  server.on("/", HTTP_GET, handleRoot);  // dashboard HTML
2  server.on("/photo", HTTP_GET, handlePhoto);  // dernier JPEG
3  server.on("/status", HTTP_GET, handleStatus);  // JSON : etat feu,
        cooldown
4  server.on("/log", HTTP_GET, handleLog);  // journal systeme
5  // Endpoints de test (force capteurs)
6  server.on("/test/red", HTTP_POST, handleTestRed);
7  server.on("/test/pir", HTTP_POST, handleTestPir);
8  server.on("/test/resetcooldown", HTTP_POST, handleTestReset);

```

Listing 4.5 – Endpoints HTTP – Serveur embarque

4.3.4 loop Radar

```

1  typedef enum { IDLE, SWEEP_ACTIVE, CAPTURE, COOLDOWN } RadarState;
2  RadarState state = IDLE;
3

```

```
4 void loop() {
5     server.handleClient();    // traitement requetes HTTP
6     unsigned long now = millis();
7
8     // Balayage servo non-bloquant (toujours actif)
9     if (now - lastServoMillis >= servoInterval) {
10         lastServoMillis = now;
11         angle += direction;
12         myServo.write(angle);
13         if (angle >= SERVO_MAX || angle <= SERVO_MIN) direction *= -1;
14     }
15
16     // FSM principale
17     switch(state) {
18         case IDLE:
19             if (isRed) state = SWEEP_ACTIVE;
20             break;
21
22         case SWEEP_ACTIVE:
23             if (!isRed) { state = IDLE; break; }
24             if (pirFlag) {
25                 pirFlag = false;
26                 if (now - lastCaptureMillis >= captureDelay) {
27                     lastCaptureMillis = now;
28                     captureInfraction();
29                     state = COOLDOWN;
30                 }
31             }
32             break;
33
34         case COOLDOWN:
35             if (now - lastCaptureMillis >= captureDelay)
36                 state = isRed ? SWEEP_ACTIVE : IDLE;
37             break;
38     }
39 }
```

4.4 Schema de câblage

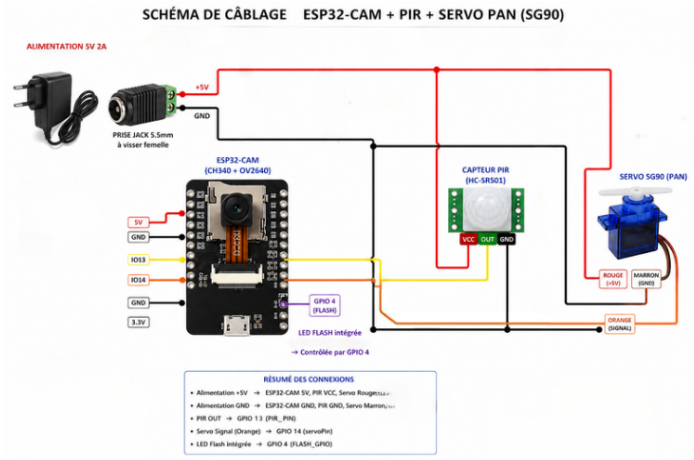


FIGURE 4.1 – Schema de câblage – Radar (ESP32-CAM + PIR HC-SR501 + Servo SG90)

TABLE 4.4 – Câblage complet – Module Radar

Composant	Broche	GPIO	Remarque
PIR HC-SR501	VCC	5 V (alim.)	
PIR HC-SR501	OUT	GPIO13	éviter GPIO15
PIR HC-SR501	GND	GND commun	
Servo SG90	VCC rouge	5 V source externe	
Servo SG90	PWM jaune	GPIO14	
Servo SG90	GND marron	GND commun	
Flash LED	anode	GPIO4	

4.5 State Machine – Radar

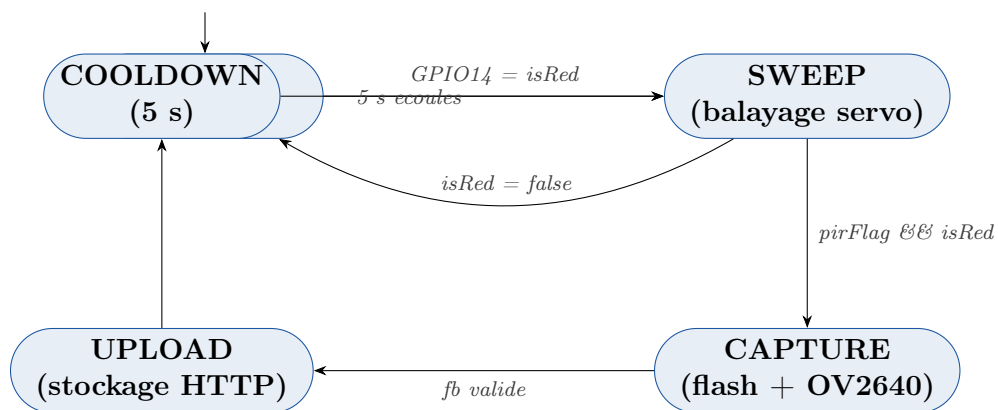


FIGURE 4.2 – FSM

5. Module 3 – Manette

5.1 Presentation et role

La **Manette** est le module de controle du robot voiture. Un ESP32 lit les valeurs analogiques du joystick (axes X et Y) et l'etat du bouton-clic, puis transmet ces donnees via **ESP-NOW** au robot voiture a 20 trames par seconde.

5.2 Capteurs utilises

5.2.1 Module Joystick analogique (VRX, VRY, SW)

Capteur : Le joystick combine deux *potentiometres lineaires* (axes X et Y) et un *contact momentane* (bouton SW). Les axes sont lus par l'**ADC 12 bits** de l'ESP32 (plage 0–4095).

TABLE 5.1 – Caracteristiques Joystick analogique – Manette

Parametre	Valeur
Type axes X/Y	Potentiometre rotatif 10 kΩ
Resolution ADC	12 bits (0–4095) sur ESP32
Tension reference	3.3 V (VREF ADC ESP32)
Position neutre	≈ 2048 (mi-course)
GPIO VRX	GPIO34 (ADC1_CH6, entree seule)
GPIO VRY	GPIO35 (ADC1_CH7, entree seule)
Type bouton SW	Contact NO, mode INPUT_PULLUP
GPIO SW	GPIO25
Frequence lecture	20 Hz (toutes les 50 ms)

5.3 Code de manette

```
1 #include <esp_now.h>
2 #include <WiFi.h>
3
4 // ===== STRUCTURE =====
5 typedef struct struct_message {
6     int x;
```

```

7   int y;
8   int button;
9 } struct_message;
10
11 struct_message data;
12
13 // ===== MAC ADDRESS OF ROBOT =====
14 uint8_t receiverAddress[] = {
15     0x8C,
16     0xAA,
17     0xB5,
18     0x0D,
19     0x10,
20     0x83
21 };
22
23 // ===== JOYSTICK PINS =====
24 #define VRX 34
25 #define VRY 35
26 #define SW 25
27
28 // ===== CALLBACK =====
29 void OnDataSent(const uint8_t *mac_addr,
30                 esp_now_send_status_t status) {
31
32     Serial.print("Send_Status: ");
33
34     if (status == ESP_NOW_SEND_SUCCESS)
35         Serial.println("Success");
36     else
37         Serial.println("Fail");
38 }
39 void setup() {
40     Serial.begin(115200);
41     pinMode(SW, INPUT_PULLUP);
42     // WIFI
43     WiFi.mode(WIFI_STA);
44     // VERY IMPORTANT
45     WiFi.setSleep(false);
46     // ESP NOW
47     if (esp_now_init() != ESP_OK) {
48         Serial.println("ESP-NOW Init Failed");
49         return;
50     }
51     esp_now_register_send_cb(OnDataSent);
52     // PEER CONFIG
53     esp_now_peer_info_t peerInfo;
54     memcpy(peerInfo.peer_addr, receiverAddress, 6);
55     peerInfo.channel = 0;
56     peerInfo.encrypt = false;
57     if (esp_now_add_peer(&peerInfo) != ESP_OK) {

```

```
58     Serial.println("Peer_Add_Failed");
59     return;
60 }
61 Serial.println("Controller_Ready");
62 }
63 void loop() {
64     data.x = analogRead(VRX);
65     data.y = analogRead(VRY);
66     data.button = digitalRead(SW);
67     esp_now_send(
68         receiverAddress,
69         (uint8_t *) &data,
70         sizeof(data)
71     );
72
73     // FAST REFRESH
74     delay(10);
75 }
```

Listing 5.1 – Lecture joystick et envoi ESP-NOW – manette.ino

5.3.1 Explication du code

Fonction OnDataSent()

Rôle et principe :

Affiche l'état de transmission des données envoyées au robot via le protocole ESP-NOW.

Objectif :

Vérifier si l'envoi des données a réussi ou échoué.

Fonction setup()

Rôle et principe :

Initialise la manette en configurant le WiFi, le protocole ESP-NOW, les pins du joystick et la connexion avec le robot.

Objectif :

Préparer la communication et les périphériques avant le démarrage.

Fonction loop()

Rôle et principe :

Lit continuellement les valeurs du joystick (X, Y et bouton) puis envoie les données au robot via ESP-NOW.

Objectif :

Assurer le contrôle temps réel du robot.

TABLE 5.2 – Cablage – Manette ESP32

Joystick	Broche	GPIO ESP32	Remarque
VRX	OUT	GPIO34	ADC seul (pas de sortie possible)
VRX	OUT	GPIO35	ADC seul
SW	OUT	GPIO25	INPUT_PULLUP
VCC	VCC	3.3 V	Alimentation joystick
GND	GND	GND	Masse commune

5.4 State Machine – Manette

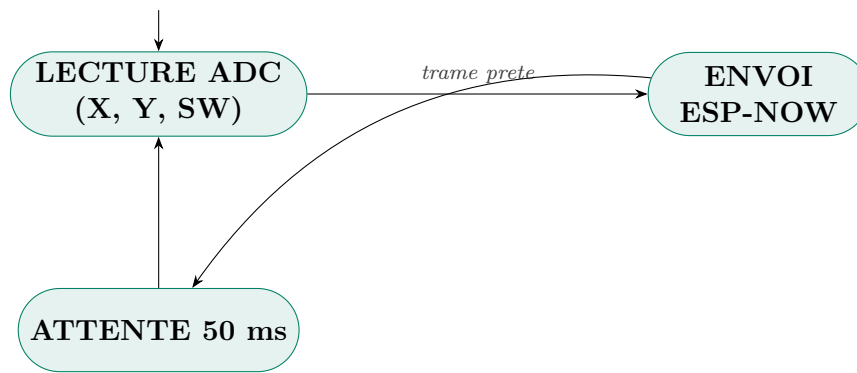


FIGURE 5.1 – FSM

6. Module 4 – Robot Voiture

6.1 Presentation et role

Le **Robot Voiture** est pilote a distance via l'ESP8266 qui recoit les trames joystick par ESP-NOW et commande un pont en H **L298N** gerant deux moteurs DC en propulsion differentielle. Un systeme de timeout de securite arrete le robot si le signal est perdu plus d'une seconde.

6.2 Capteurs utilises

Le module voiture ne comporte pas de capteurs physiques propres : il agit entierement sur la base des donnees recues de la manette (capteur de porte).

6.3 Actionneurs utilises

6.3.1 Pont en H L298N – Commande moteurs DC

Actionneur : Le L298N est un **double pont en H** permettant de commander deux moteurs DC en sens avant/arriere et en vitesse variable (PWM). Il supporte jusqu'a 2 A par canal et 46 V. La vitesse est controlee par **PWM** sur les pins ENA/ENB, et le sens par les combinaisons logiques IN1/IN2 (moteur gauche) et IN3/IN4 (moteur droit).

TABLE 6.1 – Caracteristiques L298N et logique de commande

Parametre	Valeur
Courant max / canal	2 A (4 A crete)
Tension moteur	5-46 V
Logique commande	3.3/5 V (compatible NodeMCU direct)
Commande vitesse	PWM sur ENA (gauche) et ENB (droit)
Commande sens	IN1/IN2 (gauche) : HIGH/LOW = avant, LOW/HIGH = arriere
Zone morte logicielle	speed < 50 → arret (evite vibration a l'arret)
Vitesse max logicielle	450 / 1023 (PWM limite pour controle doux)

6.4 Code voiture

```

1  #include <ESP8266WiFi.h>
2  #include <espnw.h>
3
4  // ===== PINS =====
5  #define IN1 D1
6  #define IN2 D2
7  #define IN3 D5
8  #define IN4 D6
9  #define ENA D7
10 #define ENB D8
11
12 // ===== SETTINGS =====
13 #define MAX_MOTOR_SPEED 250
14 #define SIGNAL_TIMEOUT 300
15
16 unsigned long lastRecvTime = 0;
17 bool robotActif = false;
18 bool lastButtonState = HIGH;
19
20 // ===== DATA STRUCTURE =====
21 typedef struct struct_message {
22     int x;
23     int y;
24     int button;
25 } struct_message;
26
27 struct_message receiverData;
28
29 // ===== STOP MOTORS =====
30 void stopMotors() {
31
32     digitalWrite(IN1, LOW);
33     digitalWrite(IN2, LOW);
34
35     digitalWrite(IN3, LOW);
36     digitalWrite(IN4, LOW);
37
38     analogWrite(ENA, 0);
39     analogWrite(ENB, 0);
40 }
41
42 // ===== MOTOR CONTROL =====
43 void rotateMotor(int rightMotorSpeed,
44                 int leftMotorSpeed) {
45
46     // HARD DEADZONE
47     if (abs(rightMotorSpeed) < 80)
48         rightMotorSpeed = 0;

```

```

49
50 if (abs(leftMotorSpeed) < 80)
51     leftMotorSpeed = 0;
52
53 // ----- RIGHT MOTOR -----
54 if (rightMotorSpeed > 0) {
55     digitalWrite(IN3, HIGH);
56     digitalWrite(IN4, LOW);
57 } else if (rightMotorSpeed < 0) {
58     digitalWrite(IN3, LOW);
59     digitalWrite(IN4, HIGH);
60 } else {
61     digitalWrite(IN3, LOW);
62     digitalWrite(IN4, LOW);
63 }
64 // ----- LEFT MOTOR -----
65 if (leftMotorSpeed > 0) {
66     digitalWrite(IN1, HIGH);
67     digitalWrite(IN2, LOW);
68 } else if (leftMotorSpeed < 0) {
69     digitalWrite(IN1, LOW);
70     digitalWrite(IN2, HIGH);
71
72 } else {
73
74     digitalWrite(IN1, LOW);
75     digitalWrite(IN2, LOW);
76 }
77
78 analogWrite(ENA, abs(leftMotorSpeed));
79 analogWrite(ENB, abs(rightMotorSpeed));
80 }
81 // ===== MOVEMENT =====
82 void throttleAndSteeringMovements() {
83
84     int xVal = receiverData.x;
85     int yVal = receiverData.y;
86     // LARGE DEADZONE
87     if (xVal > 1500 && xVal < 2600)
88         xVal = 2048;
89     if (yVal > 1500 && yVal < 2600)
90         yVal = 2048;
91     // FORCE STOP AT CENTER
92     if (xVal == 2048 && yVal == 2048) {
93         stopMotors();
94         return;
95     }
96     int throttle = map(
97         yVal,
98         0,
99         4095,

```

```

100         -MAX_MOTOR_SPEED ,
101         MAX_MOTOR_SPEED
102     );
103     int steering = map(
104         xVal ,
105         0 ,
106         4095 ,
107         -MAX_MOTOR_SPEED ,
108         MAX_MOTOR_SPEED
109     );
110
111     int leftMotorSpeed = throttle + steering;
112     int rightMotorSpeed = throttle - steering;
113
114     leftMotorSpeed = constrain(
115         leftMotorSpeed ,
116         -MAX_MOTOR_SPEED ,
117         MAX_MOTOR_SPEED
118     );
119
120     rightMotorSpeed = constrain(
121         rightMotorSpeed ,
122         -MAX_MOTOR_SPEED ,
123         MAX_MOTOR_SPEED
124     );
125
126     rotateMotor(rightMotorSpeed ,
127         leftMotorSpeed);
128 }
129
130 // ===== RECEIVE CALLBACK =====
131 void OnDataRecv(uint8_t * mac ,
132     uint8_t *incomingData ,
133     uint8_t len) {
134
135     memcpy(&receiverData ,
136         incomingData ,
137         sizeof(receiverData));
138
139     lastRecvTime = millis();
140
141     // BUTTON TOGGLE
142     if (receiverData.button == LOW &&
143         lastButtonState == HIGH) {
144
145         robotActif = !robotActif;
146
147         if (!robotActif)
148             stopMotors();
149     }
150

```



```

151     lastButtonState = receiverData.button;
152 }
153
154 // ===== SETUP =====
155 void setup() {
156     Serial.begin(115200);
157     pinMode(IN1, OUTPUT);
158     pinMode(IN2, OUTPUT);
159     pinMode(IN3, OUTPUT);
160     pinMode(IN4, OUTPUT);
161     pinMode(ENA, OUTPUT);
162     pinMode(ENB, OUTPUT);
163     stopMotors();
164     WiFi.mode(WIFI_STA);
165     WiFi.disconnect();
166     WiFi.setSleepMode(WIFI_NONE_SLEEP);
167     if (esp_now_init() == 0) {
168         esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
169         esp_now_register_recv_cb(OnDataRecv);
170         Serial.println("Robot Ready");
171     }
172 }
173 // ===== LOOP =====
174 void loop() {
175     // FAILSAFE
176     if (millis() - lastRecvTime > SIGNAL_TIMEOUT) {
177         stopMotors();
178         receiverData.x = 2048;
179         receiverData.y = 2048;
180         return;
181     }
182     if (!robotActif) {
183         stopMotors();
184         return;
185     }
186     throttleAndSteeringMovements();
187 }

```

Listing 6.1 – code voiture

6.4.1 Explication du code

Fonction stopMotors()

Rôle et principe :

Arrête complètement les deux moteurs du robot en mettant les pins de direction à LOW et la vitesse PWM à 0.

Objectif :

Assurer un arrêt rapide et sécurisé du robot.

Fonction rotateMotor()

Rôle et principe :

Contrôle la direction et la vitesse des moteurs. La fonction applique une zone morte pour éliminer les mouvements parasites, définit le sens de rotation des moteurs et ajuste leur vitesse grâce au signal PWM.

Objectif :

Permettre un pilotage précis des moteurs.

Fonction throttleAndSteeringMovements()

Rôle et principe :

Convertit les données du joystick en mouvements du robot. Elle lit les valeurs X et Y, applique une zone morte centrale, puis calcule les vitesses des moteurs gauche et droit.

Objectif :

Commander les déplacements du robot (avance, recul et rotation).

Fonction OnDataRecv()

Rôle et principe :

Reçoit les données envoyées par la manette via ESP-NOW, met à jour les informations de contrôle et gère l'activation ou l'arrêt du robot.

Objectif :

Assurer la communication sans fil du système.

Fonction setup()

Rôle et principe :

Initialise les pins, la communication série, le WiFi et le protocole ESP-NOW au démarrage du système.

Objectif :

Préparer le robot avant son fonctionnement.

Fonction loop()

Rôle et principe :

Exécute en continu le programme principal, vérifie la perte de signal et contrôle les mouvements du robot.

Objectif :

Garantir un fonctionnement continu et sécurisé.

6.5 Schema de câblage

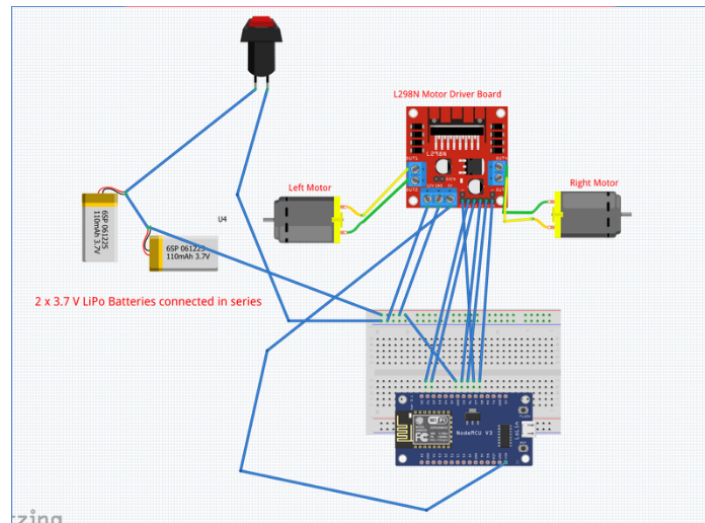


FIGURE 6.1 – Schema de câblage – Robot Voiture (ESP8266 + L298N + Moteurs DC)

TABLE 6.2 – Câblage – Robot Voiture ESP8266 / L298N

Signal	L298N	GPIO NodeMCU	Remarque
Dir. G-avant	IN1	D1 (GPIO5)	Moteur gauche avant
Dir. G-arriere	IN2	D2 (GPIO4)	Moteur gauche arriere
Dir. D-avant	IN3	D5 (GPIO14)	Moteur droit avant
Dir. D-arriere	IN4	D6 (GPIO12)	Moteur droit arriere
Vitesse G	ENA	D7 (GPIO13)	PWM analogWrite
Vitesse D	ENB	D8 (GPIO15)	PWM analogWrite

6.6 State Machine – Robot Voiture

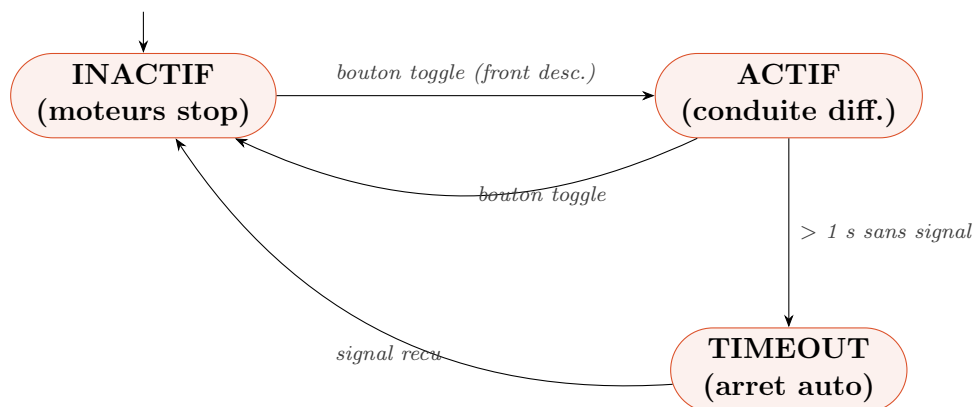


FIGURE 6.2 – FSM

7. Module 5 – Parking

7.1 Presentation et rôle

Le module **Parking** gere l'accès a un parking de 4 places via deux capteurs ultrasons (entree S1 et sortie S2), un servomoteur comme barriere, un afficheur LCD I2C pour le nombre de places disponibles, et un buzzer d'alarme. Il fonctionne de maniere **entierement autonome** (pas de WiFi).

7.2 Capteurs utilisés

7.2.1 Capteur ultrason HC-SR04 (x2)

Capteur : Le HC-SR04 mesure la distance par **temps de vol ultrasonique** : une impulsion de 10 μs sur TRIG declenche l'emission de 8 cycles a 40 kHz. Le signal ECHO reste HIGH pendant une duree proportionnelle a la distance :

$$d \text{ (cm)} = \frac{t_{echo} \text{ (}\mu s\text{)} \times 0.034}{2}$$

Le facteur 2 correspond au trajet aller-retour de l'onde.

TABLE 7.1 – Caracteristiques HC-SR04

Parametre	Valeur
Principe	Temps de vol ultrasonique (40 kHz)
Tension alim.	5 V DC
Courant	15 mA
Portee	2 cm – 4 m
Resolution	3 mm
Angle emission	15°
Impulsion TRIG	10 μs HIGH
Signal ECHO	duree proportionnelle a la distance
Seuil detection	≤ 14 cm = vehicule present
S1 (entree)	TRIG : D2 / ECHO : D3
S2 (sortie)	TRIG : D6 / ECHO : D7

1 // Declenchement ultrason S1

```

2 digitalWrite(trigPin, LOW); delayMicroseconds(2);
3 digitalWrite(trigPin, HIGH); delayMicroseconds(10);
4 digitalWrite(trigPin, LOW);
5
6 // Lecture echo (timeout 30 ms pour eviter blocage)
7 duration = pulseIn(echoPin, HIGH, 30000);
8 distance = duration * 0.034 / 2;
9
10 bool s1 = (distance > 0 && distance <= 14); // vehicule detecte

```

Listing 7.1 – Mesure de distance par ultrason HC-SR04

7.3 Actionneurs utilisés

7.3.1 Servo – Barriere d'accès

Actionneur : Le servomoteur joue le rôle de *barrière mécanique* : position 90° = ferme (bloque le passage), 180° = ouvert. Il se ferme automatiquement 4s après ouverture via un timer logiciel (*millis()*).

TABLE 7.2 – Caracteristiques Servo barriere – Parking

Parametre	Valeur
Type	Servomoteur a position
GPIO signal	D10 (pin 10 Arduino, PWM)
Position fermee	90°
Position ouverte	180°
Temporisation	4000 ms apres ouverture

7.3.2 Afficheur LCD I2C 16x2

Actionneur : L'afficheur LCD 16x2 utilise le protocole *I2C* via un module d'adaptation PCF8574. Seulement 2 fils (SDA, SCL) suffisent pour afficher le nombre de places disponibles en temps reel.

TABLE 7.3 – Caracteristiques LCD I2C

Parametre	Valeur
Dimensions	16 colonnes × 2 lignes
Interface	I2C via PCF8574
Adresse I2C	0x27 (par defaut)
SDA	A4 (Arduino UNO)
SCL	A5 (Arduino UNO)
Librairie	LiquidCrystal_I2C
Contenu ligne 0	“Places libres :”
Contenu ligne 1	valeur entiere (0–4)

7.3.3 Buzzer – Alarme parking complet

Actionneur : Le buzzer piezoelectrique emit une tonalite de 1 kHz pendant 400 ms lorsque le parking est plein et qu’un vehicule tente d’entrer (*tone()* / *noTone()* Arduino).

7.3.4 LED indicateur

Actionneur : La LED (GPIO D5/pin 5 Arduino, PWM) s’allume des qu’un vehicule est detecte par S1 ou S2 (*s1 || s2*), indiquant visuellement une detection active.

```

1 bool s1 = (distance > 0 && distance <= 14);
2 bool s2 = (distance2 > 0 && distance2 <= 14);
3
4 digitalWrite(ledPin, s1 || s2);    // LED ON si vehicule detecte

```

Listing 7.2 – Commande LED selon detection ultrason

7.4 Schema de câblage

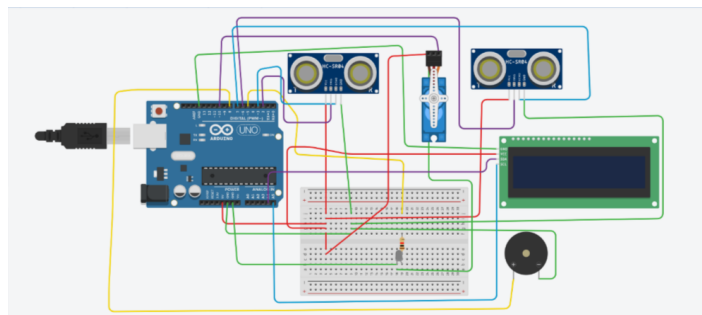


FIGURE 7.1 – Schema de câblage – Module Parking (Arduino UNO + HC-SR04 x2 + Servo + LCD + Buzzer + LED)

TABLE 7.4 – Câblage complet – Module Parking Arduino UNO

Composant	Broche	Pin Arduino	Remarque
HC-SR04 S1 (entree)	TRIG	D2	Pulse 10 μ s
HC-SR04 S1	ECHO	D3	pulseIn()
HC-SR04 S2 (sortie)	TRIG	D6	Pulse 10 μ s
HC-SR04 S2	ECHO	D7	pulseIn()
LED indicateur	anode	D5 (PWM)	220 Ω serie
Buzzer	signal	D8	tone()
Servo barriere	signal	D10 (PWM)	90/180°
LCD I2C (SDA)	SDA	A4	Adresse 0x27
LCD I2C (SCL)	SCL	A5	

7.5 Logique de detection entree/sortie et State Machine

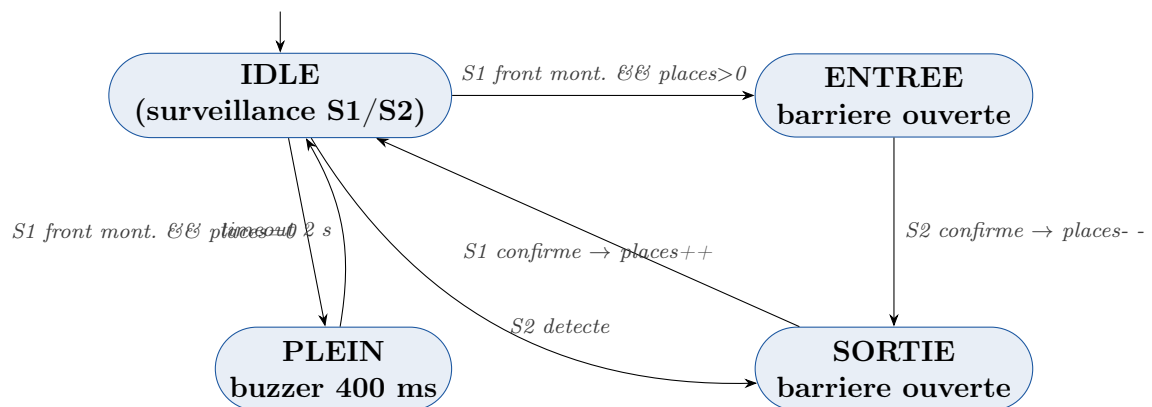


FIGURE 7.2 – FSM – Module Parking (Arduino UNO)

7.6 Code complet

```

1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <Servo.h>
4
5 const int trigPin = 2;
6 const int echoPin = 3;
7 const int trigPin2 = 6;
8 const int echoPin2 = 7;
9 int ledPin= 5;
10 int buzzpin =8;
11 Servo barriere;
12 int servoPin = 10;
13
14 int angleOpen = 180;
15 int angleClose = 90;

```

```
16
17 unsigned long openTime = 0;
18 bool barrierOpen = false;
19 int barrierDelay = 5500;
20
21 long duration;
22 int distance;
23 long duration2;
24 int distance2;
25
26 int places= 1;
27 int prev_s=0;
28 int nbr_max= 2;
29
30 // detect new arrival at S1
31 bool s1_prev = false;
32 bool s2_prev = false;
33
34 // Timeout: if sequence not completed in 8s, reset
35 unsigned long sequenceStart = 0;
36 const unsigned long SEQUENCE_TIMEOUT = 8000;
37
38 LiquidCrystal_I2C lcd(0x27, 16, 2);
39
40 // ===== SETUP =====
41 void setup() {
42     Serial.begin(9600);
43     pinMode(ledPin, OUTPUT);
44     pinMode(trigPin, OUTPUT);
45     pinMode(echoPin, INPUT);
46     pinMode(trigPin2, OUTPUT);
47     pinMode(echoPin2, INPUT);
48     pinMode(buzzpin, OUTPUT);
49
50     barriere.attach(servoPin);
51     barriere.write(angleClose);
52
53     lcd.init();
54     lcd.backlight();
55 }
56
57 // ===== LOOP =====
58 void loop() {
59
60     // ===== S1 =====
61     digitalWrite(trigPin, LOW);
62     delayMicroseconds(2);
63     digitalWrite(trigPin, HIGH);
64     delayMicroseconds(10);
65     digitalWrite(trigPin, LOW);
66
```



```

67 duration = pulseIn(echoPin, HIGH, 30000);
68 distance = (duration == 0) ? 999 : duration * 0.034 / 2;
69
70 delay(50);
71
72 // ===== S2 =====
73 digitalWrite(trigPin2, LOW);
74 delayMicroseconds(2);
75 digitalWrite(trigPin2, HIGH);
76 delayMicroseconds(10);
77 digitalWrite(trigPin2, LOW);
78
79 duration2 = pulseIn(echoPin2, HIGH, 30000);
80 distance2 = (duration2 == 0) ? 999 : duration2 * 0.034 / 2;
81
82 bool s1 = (distance > 0 && distance <= 14);
83 bool s2 = (distance2 > 0 && distance2 <= 14);
84
85 digitalWrite(ledPin, s1 || s2);
86
87 // detect NEW arrival at S1
88 bool s1_rising = s1 && !s1_prev;
89 bool s2_rising = s2 && !s2_prev;
90 s1_prev = s1;
91 s2_prev = s2;
92
93 lcd.setCursor(0, 0);
94 lcd.print("Places_librés:");
95
96 // ===== SEQUENCE TIMEOUT prevents permanent lock =====
97 if (prev_s != 0 && millis() - sequenceStart > SEQUENCE_TIMEOUT) {
98     prev_s = 0;
99 }
100
101 if (prev_s == 0) {
102
103     // lel louel s1
104     if (s1_rising) {
105
106         // pprking maabiii
107         if (places == 0) {
108
109             tone(buzzpin, 1000);
110             delay(400);
111             noTone(buzzpin);
112
113             lcd.setCursor(0, 0);
114             lcd.print("PARKING FULL");
115             lcd.setCursor(0, 1);
116             lcd.print("PARKING FULL");
117

```

```
118         delay(2000);
119
120         lcd.setCursor(0, 0);
121         lcd.print("Places libres:");
122         lcd.setCursor(0, 1);
123         lcd.print(" ");
124         lcd.setCursor(0, 1);
125         lcd.print(places);
126     }
127
128     // il ya de places dans le parking
129     else {
130         prev_s = 1;
131         sequenceStart = millis();
132         barriere.write(angleOpen);
133         openTime = millis();
134         barrierOpen = true;
135     }
136 }
137
138 // ken louel s2 EXIT
139 else if (s2_rising) {
140     prev_s = 2;
141     sequenceStart = millis();
142     barriere.write(angleOpen);
143     openTime = millis();
144     barrierOpen = true;
145 }
146 }
147
148 // entr e CONFIRMED
149 if (prev_s == 1) {
150     if (s2) {
151         if (places > 0 ) {
152             places--;
153             prev_s = 0;
154         }
155     }
156 }
157
158 // sortie CONFIRMED
159 if (prev_s == 2) {
160     if (s1) {
161         if (places < nbr_max )
162             places++;
163         prev_s = 0;
164     }
165 }
166
167 // fermeture barriere
168 if (barrierOpen && millis() - openTime > barrierDelay) {
```

```
169     barriere.write(angleClose);
170     barrierOpen = false;
171 }
172
173 // LCD
174 lcd.setCursor(0, 1);
175 lcd.print(places);
176
177 delay(200);
178 }
```

Listing 7.3 – Logique entrée/sortie et gestion des places – parking.ino

8. Conclusion

Ce rapport présente l'architecture complète du système **Smart Street**, composé de cinq modules embarqués illustrant les concepts fondamentaux du cours **Capteurs & Actionneurs** :

1. **Feu Tricolore (ESP8266)** : lecture analogique LDR (photoresistance), détection numérique bouton piéton (INPUT_PULLUP), commande LEDs. Emit l'état du feu via ESP-NOW.
2. **Radar Intelligent (ESP32-CAM)** : detection IR passive (HC-SR501) en interruption, commande servo SG90 en PWM pour balayage panoramique, déclenchement caméra OV2640 sur condition logique (feu + PIR), serveur HTTP embarqué.
3. **Manette (ESP32)** : acquisition analogique 12 bits joystick (deux potentiometres VRX/VRX) et bouton numérique SW, emission ESP-NOW a 20 Hz.
4. **Robot Voiture (ESP8266 + L298N)** : reception ESP-NOW, commande pont en H (direction + PWM vitesse), conduite différentielle avec zone morte, timeout sécurité 1 s.
5. **Parking (Arduino UNO)** : mesure de distance ultrasonique HC-SR04 (temps de vol), servomoteur barrière, afficheur LCD I2C, buzzer alarme, logique d'entrée/sortie a deux capteurs.

L'ensemble du système illustre la chaîne complète **Capteur** → **Traitement** → **Actionneur** dans un contexte distribué, avec des contraintes temps-reel gérées par FSM et loop non-bloquante.